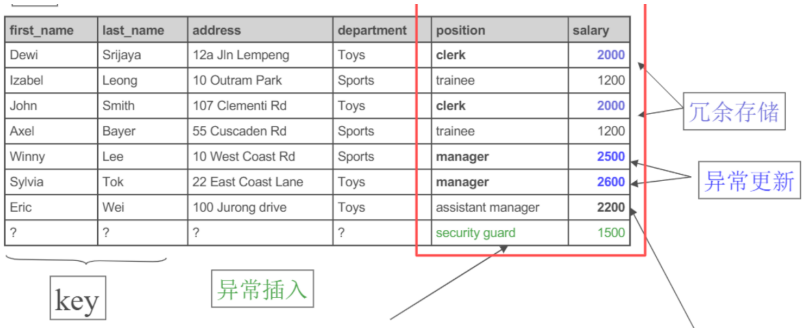
数据库系统笔记：中大 2022 人工智能学院课程

9. 关系型数据库设计：3NF

数据库异常行为包括函数依赖下的冗余存储、异常更新、异常插入、异常删除，如下图（position→salary）。



first_name	last_name	address	department	position	salary
Dewi	Srijaya	12a Jln Lempeng	Toys	clerk	2000
Izabel	Leong	10 Outram Park	Sports	trainee	1200
John	Smith	107 Clementi Rd	Toys	clerk	2000
Axel	Bayer	55 Cuscaden Rd	Sports	trainee	1200
Winny	Lee	10 West Coast Rd	Sports	manager	2500
Sylvia	Tok	22 East Coast Lane	Toys	manager	2600
Eric	Wei	100 Jurong drive	Toys	assistant manager	2200
?	?	?	?	security guard	1500

图 1: 数据库异常行为

规范化是将关系型数据库分解的过程。$\mathbf{R} = R_1, R_2, \ldots, R_n$。

- 要求分解无损、依赖保留且冗余信息小。

依赖保留：FDs 存在于各个单个 relation，以便于验证 FD。

- 即，所有子表格 FD 并集的闭包与原来表格 FD 闭包一致

> 给定一组函数依赖 F，F 的闭包记作 F^+，它是所有由 F 逻辑上蕴涵的函数依赖的集合。换句话说，F^+ 包括 F 中的所有函数依赖以及可以通过 F 中的函数依赖使用推理规则推导出来的所有函数依赖。
>
> 计算一个属性集 X 关于一组函数依赖 F 的闭包 X^+ 的方法如下：
>
> 1. 初始化：设 X^+ 为 X 的初始值。
> 2. 迭代：重复以下步骤直到 X^+ 不再变化：
> - 对于每一个 F 中的函数依赖 $A \to B$，
> - 如果 $A \subseteq X^+$，则将 B 添加到 X^+ 中。
>
> 当 X^+ 不再变化时，算法结束，此时 X^+ 就是 X 在 F 下的闭包。

不满足依赖保留的分解：

- 可能会出现信息丢失、存储冗余等问题。
- 如果不能通过自然连接而恢复依赖，则出现信息丢失。
- 在下图中，R_1 和 R_2 不存在依赖 $B \to C$。
- $F_1 = \{\{A\} \to \{B\}\} \& F_2 = \{\{A\} \to \{C\}\}, (F_1 \cup F_2)^+ \neq F^+$。正确分解如图 3。

Lossless Join Decomposition：

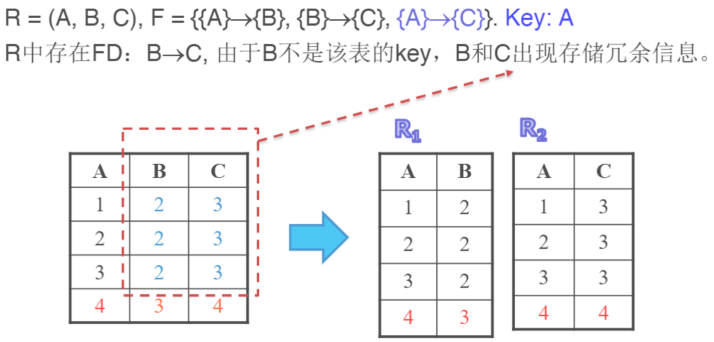


图 2: 错误分解

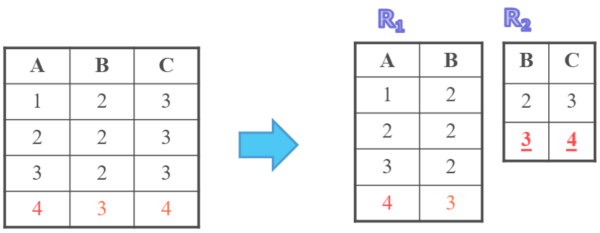


图 3: 正确分解

- 如果能利用分解后的表恢复原来的表，则该分解是无损的。
- 两个 schema 的交集属性至少构成其中一个 schema 的 key。
  - 外键必须引用另一个表的候选键。

1NF：第一范式

- 第一范式：表中所有属性的域没有多值属性。
- 根据关系模型的定义，关系表（relational tables）永远是 1NF。
  - 关系模型要求每一个属性都是原子的。

> 1NF 是不够的，不一定能满足无损、依赖保留和形式良好。
> 范式是对关系模式的规范化，不是对分解方法的选择。

2NF：第二范式

- 定义：1NF 基础上，所有非主属性完全函数依赖于任何一个候选键。
  - 2NF 确保所有非主属性完全依赖于任何一个候选键，而不是部分依赖于候选键的一部分。即，非主属性必须完全依赖于候选码。
  - 2NF 通过消除部分函数依赖，可以减少数据冗余，提高数据的一致性和完整性。

- 形式化：R 是在 2NF 中当且仅当对于每个函数依赖 $X \to \{A\}$ 在 F^+ 中，至少满足以下三个条件的其中一条：
  - $A \in X$（函数依赖是平凡的）：自己依赖自己，无函数依赖
  - X 不是某个候选键的真子集：被依赖属性非为部分候选键
  - A 是一个主属性
- 向 2NF 的转换：
  - 如果存在非主属性对候选码的部份依赖，则这个属性和主关键字这一部分应该分离开作为一个新表。

假设关系模式 $R(A, B, C, D)$，(A, B) 是主键，函数依赖集为 $F = \{A \to C, (A, B) \to D\}$。
- 主键是 (A, B)。
- 非主属性是 C 和 D。
- 函数依赖 $A \to C$ 表明 C 部分依赖于主键 (A, B) 的一部分 A。
- 函数依赖 $(A, B) \to D$ 表明 D 完全依赖于主键 (A, B)。

结论：关系模式 R 不满足 2NF，因为存在部分函数依赖 $A \to C$。

转换为 2NF：
- 分离成为两个关系模式 $R_1(A, C)$ 和 $R_2(A, B, D)$。
- R_1 和 R_2 的 FDs，非主属性都完全依赖于主键。

3NF：第三范式

- 定义：在 1NF 的基础上，任何非主属性不依赖于非键。
  - 理解：在 2NF 的基础上消除传递依赖。
  - 3NF 同样减少数据冗余和提高数据的一致性。
- 形式化：R 是在 3NF 中当且仅当对于每个函数依赖 $X \to \{A\}$ 在 F^+ 中，至少满足以下三个条件的其中一条：
  - $A \in X$（函数依赖是平凡的）。
  - X 是 R 的超键。（相比起 2NF 的更进一步。）
  - A 是一个主属性。
- 3NF 的分解：假设有一个关系模式 $R(A, B, C, D)$，其函数依赖集为 $F = \{A \to B, B \to C, A \to D\}$。
  - 确定候选键：计算属性闭包，发现 A 是候选键，因为 $A^+ = \{A, B, C, D\}$。
  - 识别非主属性：非主属性是 B、C 和 D。
  - 检查传递依赖：存在传递依赖 $A \to B \to C$，因为 $A \to B$ 且 $B \to C$，但 $A \not\to C$。
  - 分解关系模式：将 R 分解为两个关系模式：$R_1(A, B, D)$ 和 $R_2(B, C)$

一个值得注意的自环传递例子：
- $R = (B, C, E)$，$F = \{\{E\} \to \{B\}, \{B, C\} \to \{E\}\}$

- 找出候选键：BC 和 EC。
- 满足任何非主属性不依赖于非键。
- 该关系模式符合 3NF 范式。

BCNF：Boyce-Codd 范式

- BCNF 旨在消除非平凡的多值依赖和非主属性对候选键的部分依赖。
- 形式化：R 是在 BCNF 中当且仅当对于每个函数依赖 $X \to \{A\}$ 在 F^+ 中，至少满足以下三个条件的其中一条：
  - $A \in X$（函数依赖是平凡的）。
  - X 是 R 的超键。
- （...）

10. 内存层级和文件组织 377

11. 索引的相关概念 405

索引的引入

- ID 排序 + 二分搜索：$\log(n)$。
- ID 排序 + 索引条目：$< ID, page.No >$`[Search-key|pointer]`
  - 每个 page 的第一条记录有一个条目，从而反映条目区间。
  - 对索引使用二分查找。
  - 可以创建多级索引 Multilevel Index。
- 两种基础索引类型：顺序索引、哈希无序索引。

顺序索引

- 顺序索引也成为树索引。
- 获取一个记录所需的 page 访问次数为`tree.height+1`

主索引和辅助索引

- 主索引 Primary index：文件实际存放也是按照 search key 排序的。其基于主键或唯一标识符构建索引，具有唯一性。
- 辅助索引 Secondary index：也称为二级索引。文件的实际存放不按照 search key 排序。其基于非主属性构建索引，不具有唯一性。
  1. 当检索许多记录时，辅助索引昂贵，因为磁盘不连续寻址耗时长。
  2. 辅助索引常用于优化不涉及主键的查询。

> 对于非排序属性（如账号、余额等），可以通过额外建立索引生成二级索引，从而提高查询效率。这样可以在不需要重新组织数据的情况下，实现对这些属性的有效查询。

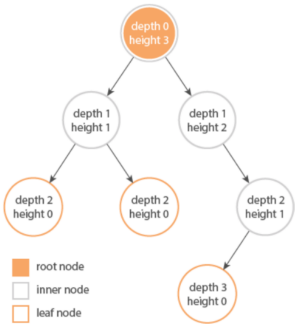


图 4: tree index

稀疏索引和密集索引

- 稀疏索引 Sparse Index：仅包含某些 search key。
  稀疏检索：找到最后一个比 K 小 key 的 index，然后向后遍历寻找 K。
- 密集索引 Dense Index：包含所有 search key。

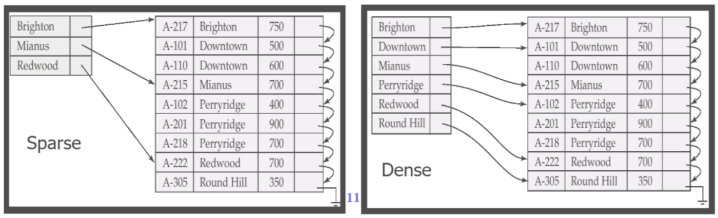


图 5: 稀疏索引和密集索引

哈希索引

- 哈希索引维护哈希函数`F(search key)=location_of_record`
- 严格来说，哈希索引是二级索引。

索引的评价指标

- 成本定义为最坏情况下的磁盘打开页数。
- 高效支持不同查询：例如，辅助索引（包括哈希）在范围检索表现很差，因为磁盘经常需要跨区寻址。
- 更新时间成本小：当文件被修改时，索引也必须更新。
- 额外存储成本小（即 index 本身的大小）：索引的大小应该比数据文件小得多。

索引的更新

- 删除：索引的删除思路符合直觉。

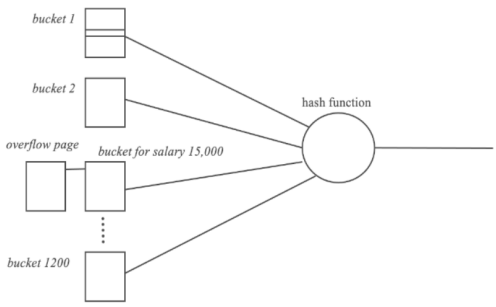


图 6: 哈希索引结构

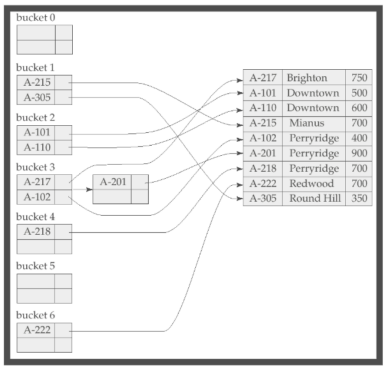


图 7: 哈希索引示例

  - 如果删除的记录为索引的唯一记录，则对应的索引也要删除。
  - 对于稀疏索引，如果删除记录的 search key 在 index 存在，则将 index 里的的记录替换为所删除的下一条记录。如果下一条记录已经在 index 中存在，则直接删除。
- 插入：同样符合直觉。
  - 对于密集索引：如果 search key 没有出现在 index 则插入。
  - 对于稀疏索引：
    1. 如果该索引为每个 page 维护有索引条目则不需要修改。
    2. 否则要创建新 page。当有新的 block（page）创建的时候，将 block 的第一个 search key 加入 index。

12. 索引进阶：B^+ 树和动态哈希

12.1 B^+ 树：

B^+ 树引入

- B^+ 树是一种特殊的平衡多路搜索树。
- B^+ 树是所有商业 DBMS 中优序索引的默认实现。

B^+ 树的主要特征

- 所有叶子节点都在同一层。
- 非叶子节点只存储索引信息。
- 叶子节点存储实际数据信息。
- 叶子节点之间有指针连接：所有的叶子节点通过指针相互连接，形成一个双向链

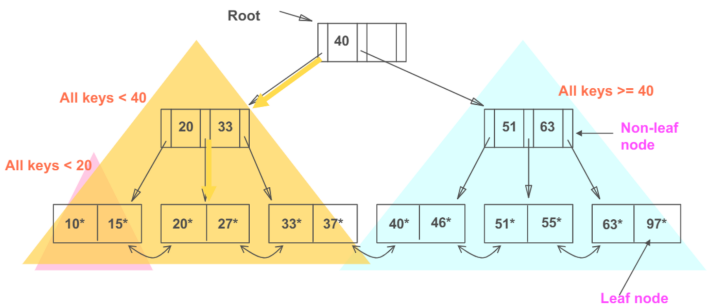


图 8: B^+ 树示意图

表。这种结构支持高效的范围查询和顺序扫描。

B^+ 树的性质

- 从根到叶子的所有路径都具有相同的长度（a balanced tree）
- n 称为扇出 fanout（每个节点最多可有拥有指针数）：
  1. 每个非叶节点都有数量在 $\lceil n/2 \rceil$ 到 n 之间的指针
  2. 每个叶节点存储数量为 $\lceil (n-1)/2 \rceil$ 和 $n-1$ 之间的数据值
  3. 值 $\lceil (n-1)/2 \rceil$ 称为 order 阶数（数据值的最小数量）

> 1. 和 2. 都是在设定了子节点下限，从而维护树的平衡。
> 特别例子：如果根不是叶节点，则至少有两个子节点。如果根是叶节点，可以有 0 到 $(n-1)$ 个数据值。

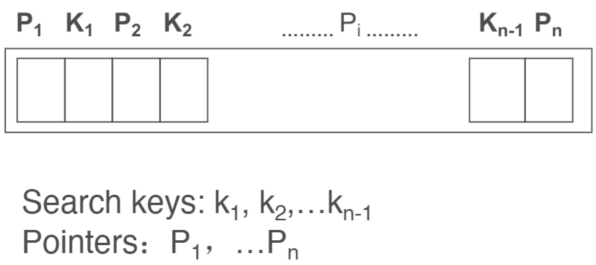


图 9: B^+ 树节点示意图

B^+ 树查询

- 从所有记录中寻找 $search_key = a$ 的记录：
  从根节点开始：
  1. 在当前节点内部寻找 P_i，满足 $K_{i-1} \leq a < k_i$。

2. 跳到 P_i 指向的子节点。重复以上步骤直到叶节点。如果在叶结点中找不到，则该树不包含 $search_key = a$ 的记录。

B^+ 树插入

- 用键值 k 进行查询，得到叶节点 L。
- 将 k 插入到 L 中：
  1. 如果 L 有空位，则插入完成。
  2. 如果 L 无空位：将 L 分裂成 L_1 和 L_2：
     (a) 将 keys 平分（奇数则左侧多一个）。
     (b) 更新 L 的父节点：插入 L_2 的最左侧数值。（中间键值往父节点插入）
     (c) 如果父节点也无空位，则由下往上递归更新节点。

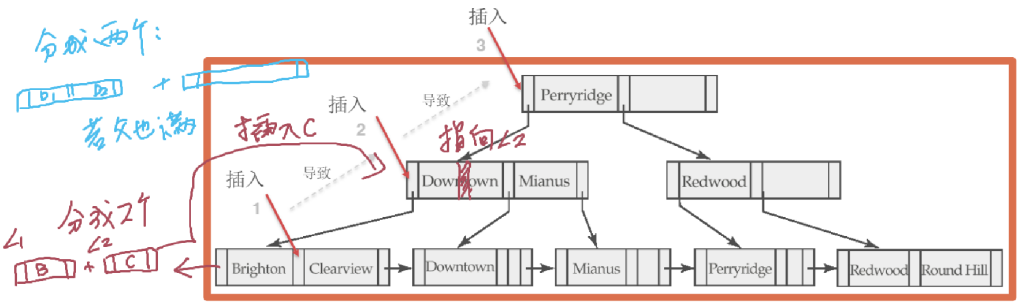


图 10: B^+ 树插入

B^+ 树删除

- 用 k 值查询，得到其所在叶节点 L。
- 将 k 从 L 中移除：
  1. 如果 L 有超过半数 key，则结束。
  2. 否则，从右侧兄弟节点索取一个 key。
     (a) 如果右侧节点少于 $\lceil (n-1)/2 \rceil + 1$ 个节点，则合并两个节点：
        – 合并后，从 L 的父节点删除指向 L 的键值。
        – 合并也可能发生连锁反映。

B^+ 树文件组织结构

- 可以用 B^+ 树索引解决索引退化问题。
- 数据文件退化问题（碎片化）用 B^+ 树来组织文件存储。要求：
  – 叶节点存储数据本身。
  – 叶节点存储的最大记录数小于非叶节点的最大指针数。
  –（上面两条都是 B^+ 树本身的要求）

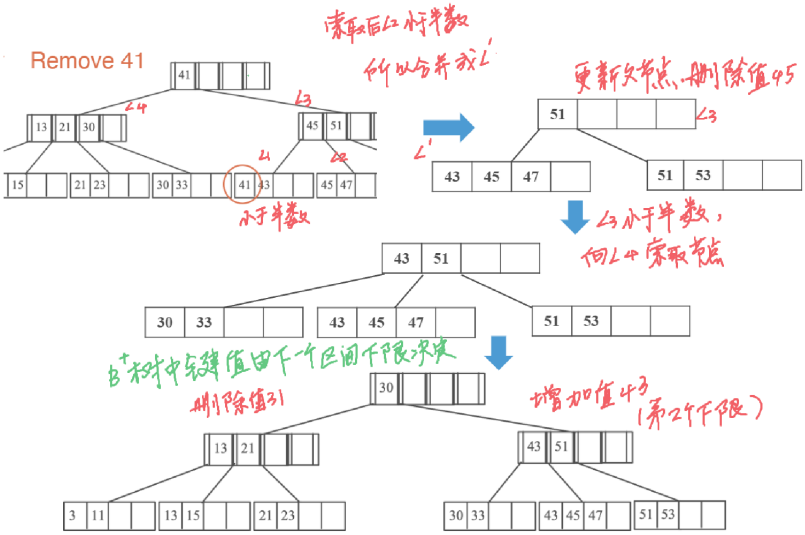


图 11: B^+ 树删除

在文件组织和数据库管理中，索引退化（Index Degradation）是指索引结构在长时间运行和频繁的插入、删除操作后，逐渐失去其优化效果，导致性能下降的问题。索引退化会影响查询性能、增加磁盘 I/O 操作次数，并可能导致其他性能问题。

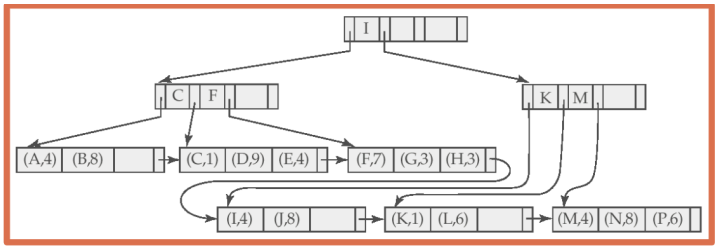


图 12: B^+ 树文件组织结构

12.2 动态哈希：

静态哈希：Buckets 数量是固定的。将搜索键值固定映射到某个 Bucket。随着数据增长，由于过多的溢出导致该 index 的性能退化。

动态哈希

- Buckets 数目可以动态调整。

- 关键：哈希结果和数据位置中间加一层指针：
  - 使用指向 buckets 的指针目录，通过将目录加倍来增加 buckets 数量（见下）。
  - 仅拆分溢出的存储 bucket。
  - 只需要对指针进行整理，成本小很多。

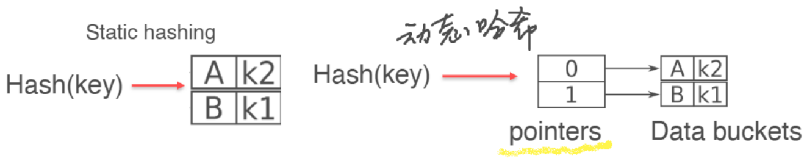


图 13: 静态与动态哈希对比

动态哈希函数的调整

- 掩码哈希：$f(value) = value\%(2^{DEPTH})$
- 通过全局深度 $GLOBAL\ DEPTH$ 和局部深度 $LOCAL\ DEPTH$ 动态分裂桶。
  - 全局深度：将一个元素映射到一个桶时需要看几位。
  - 局部深度：桶中所有元素相同尾数序列长度。
  - $Global\ depth = \max\{Local\ depth\}$
  - $Local\ depth = Global\ depth - \log_2(lines)$，其中 $lines$ 为连接线段数量。

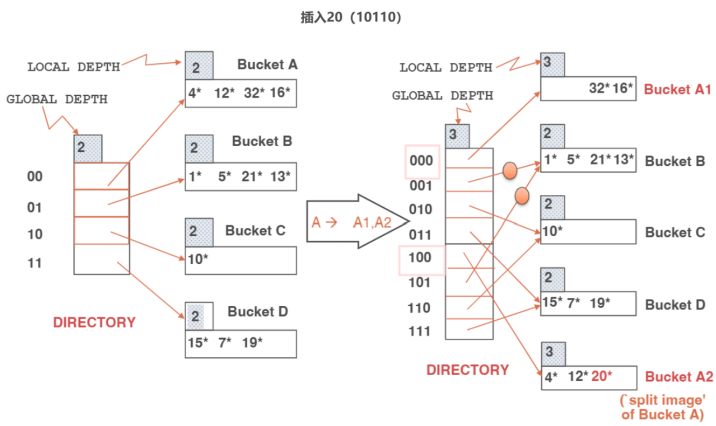


图 14: 掩码哈希动态调整

动态哈希删除

- 如果删除导致 bucket 变空，则将其与其镜像（譬如 001 和 101）合并，合并后 local depth-1。

- 当因为一系列删除操作，导致指针目录的任意指针与其镜像指针均指向同一个 bucket，则将整体指针目录缩减一半，global depth-1。

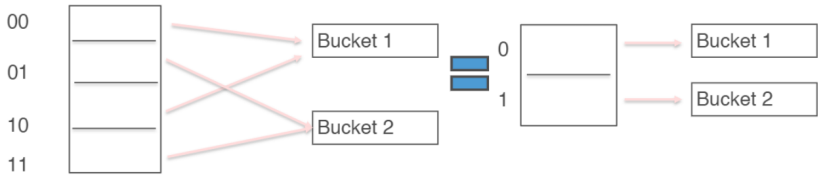


图 15: 动态哈希全局目录缩减

13. 其它索引：多键访问

多键访问引入

- 如何加快具有多个条件的查询记录？

```
Select loan-number
From account
Where branch-name<"Perryridge" and Balance = 1000
```

- 策略一：先后查询

  先检查元组的 branch-name 然后在检查 balance
- 策略二：交集查询

  先分别获取符合两种要求的指针，然后寻找指针的交集并获得记录。

位图索引

- 位图索引旨在对多个键同时进行高效查询。
- 引入：
  - 记录按照顺序编号为 $(R_0, R_1, R_2, \ldots)$，给定 n 则检索 R_n 的效率很高。
  - 仅对取值个数有限的属性特别适用。
-

附录 1：相关习题或代码